Return to *libc*

Michele LORETI
Università di Camerino





https://cybersecnatlab.it

License & Disclaimer

License Information

This presentation is licensed under the Creative Commons BY-NC License



To view a copy of the license, visit:

http://creativecommons.org/licenses/by-nc/3.0/legalcode

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

© CINI – 2021

Goal

- In this lecture we will present *return to libc* a technique for *arbitrary code execution.*

© CINI – 2021

Prerequisites

- Lecture:
  - Basic knowledge of C

© CINI – 2021

Outline

- Arbitrary Code Execution
- Return to libc

© CINI – 2021

Arbitrary Code Execution (ACE)

- The attacker's ability to execute arbitrary commands or code on a target machine or in a target process
- Can be induced by:
  - Code injection
  - Code reuse

© CINI – 2021

Code Reuse Attacks

- Code-reuse attacks are software exploits in which an attacker directs control flow through existing code with a malicious result
- Examples of such attacks are:
  - Return-to-libc
  - Return Oriented Programming (ROP)
  - Jump Oriented Programming (JOP)

© CINI – 2021

Return to libc…

- A *return-to-libc* attack is a technique that, by using a *buffer overflow*, replaces a return address with the one of another function in the process memory
- The name is related to the fact that one of functions in the *C Standard Library (libc) is used*
- This attack was spotted in 1997 by *Alexander Peslyak* (aka *Solar Designer*)

© CINI – 2021

Return to libc

- Let us consider again the following vulnerable code:

```c
#include <stdio.h>
#include <string.h>

void welcome(char *name)
{
    char buf[10];
    strcpy(buf, name);
    printf("Welcome %s\n", buf);
}

int main(int argc, char *argv[])
{
    welcome(argv[1]);
    return 0;
}
```


© CINI – 2021

Return to libc

- By performing a *return-to-libc* attack, we can execute a *shell* with the same access rights of the executed program
- The basic steps of the attack are:
  - Find the address of *system() libc* function
  - Find the address of string "*/bin/sh*"
  - Corrupt the stack and let *system()* be called with parameter */bin/sh*
- We do not need an *executable* stack!

© CINI – 2021

Return to libc…

- To find address of *system()* we can use *gdb*:

```
(gdb) file retlibc
Reading symbols from retlibc...(no debugging symbols found)...done.
(gdb) break main
Breakpoint 1 at 0x104b4
(gdb) run
Breakpoint 1, 0x000104b4 in main ()
(gdb) p system()
Too few arguments in function call.
(gdb) p system
$1 = {int (const char *)} 0xb6e909c8 <__libc_system>
```


© CINI – 2021

Return to libc…

- A similar approach can be used to find address of */bin/sh:*

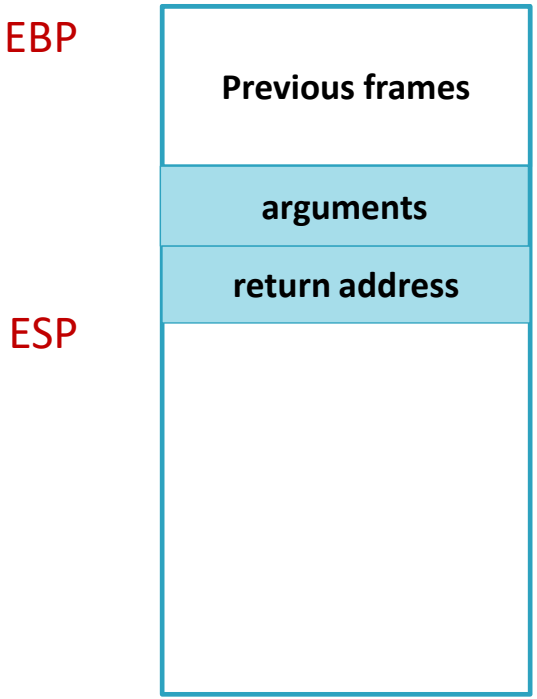
```
$1 = {int (const char *)} 0xb6e909c8 <__libc_system>
(gdb) find 0xb6e909c8, +999999999, "/bin/sh"
0xb6f83b6c
warning: Unable to access 16000 bytes of target memory at 0xb6f93574, halting se
arch.
1 pattern found.
(gdb)
```


© CINI – 2021

Stack behaviour (recall)

- Each function starts with the following instructions (in assembly) with arguments and return address on top of the stack:

```
push %ebp
mov %esp,%ebp
sub $N,%esp
```

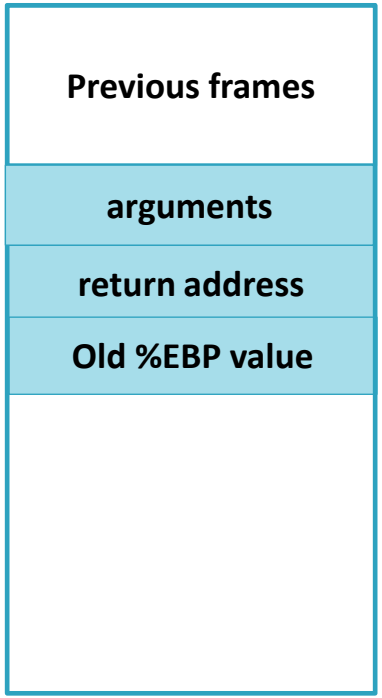


© CINI – 2021

Stack behaviour (recall)

- First, previous value of %ebp is stored:

```
push %ebp
mov %esp,%ebp
sub $N,%esp
```



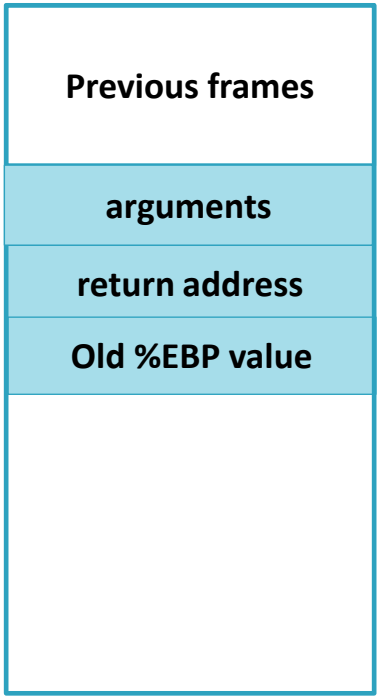
© CINI – 2021

Stack behaviour (recall)

- After that, %ebp updated to refer to the new frame:

```
push %ebp
mov %esp,%ebp
sub $N,%esp
```

EBP ESP



© CINI – 2021

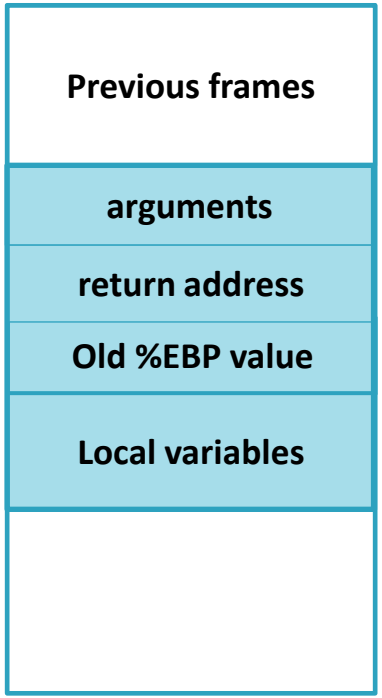
Stack behaviour (recall)

- Finally, memory for local variables is allocated:

```
push %ebp
mov %esp,%ebp
sub $N,%esp
```

EBP

ESP



© CINI – 2021

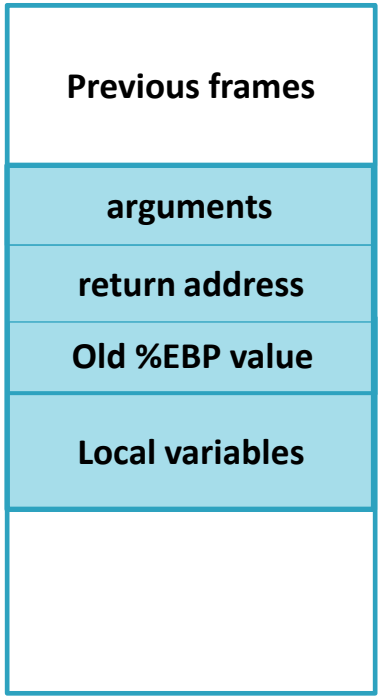
Stack behaviour (recall)

- When a function ends, the following instructions are executed:

```
mov %ebp,%esp
pop %ebp
ret
```

EBP

ESP



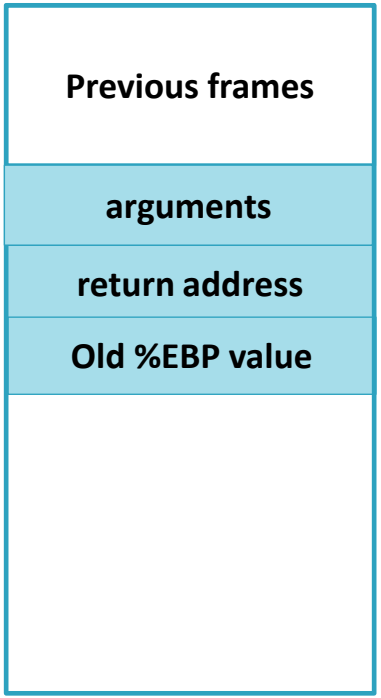
© CINI – 2021

Stack behaviour (recall)

- Register %esp is set to %ebp (local variables are *lost*):

```
mov %ebp,%esp
pop %ebp
ret
```

ESP EBP

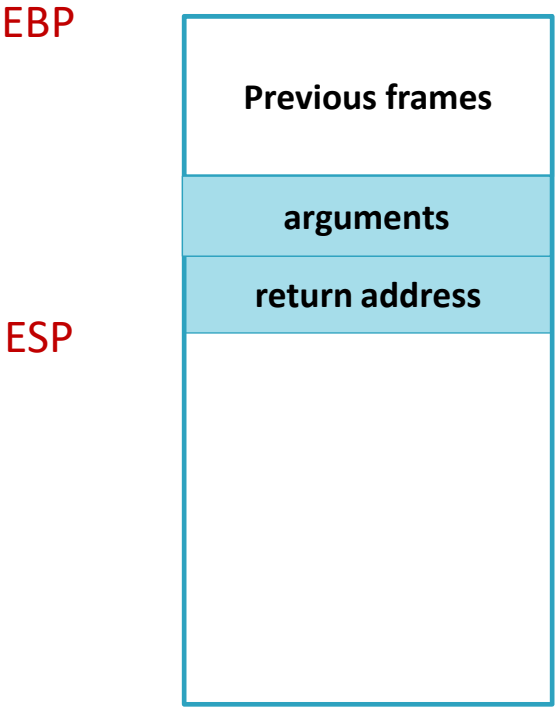


© CINI – 2021

Stack behaviour (recall)

- Previous value of %ebp is restored:

```
mov %ebp,%esp
pop %ebp
ret
```

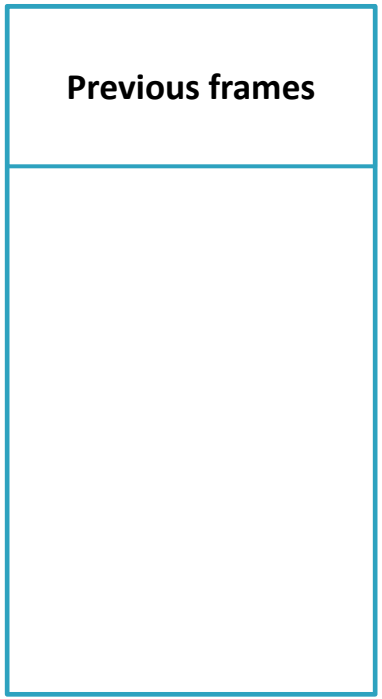


© CINI – 2021

Stack behaviour (recall)

- Execution *returns* to the *return address*:

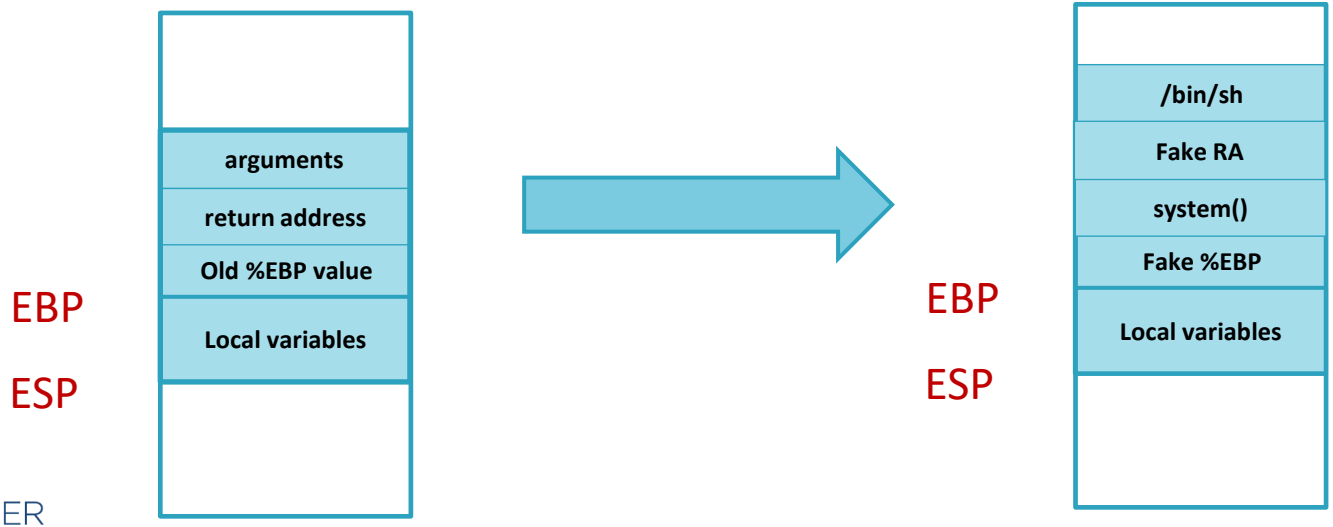
```
mov %ebp,%esp
pop %ebp
ret
```



© CINI – 2021

Return to libc in action…

- An attacker can corrupt the stack by inserting the addresses of *system()* and */bin/sh* in the *right place*:



© CINI – 2021

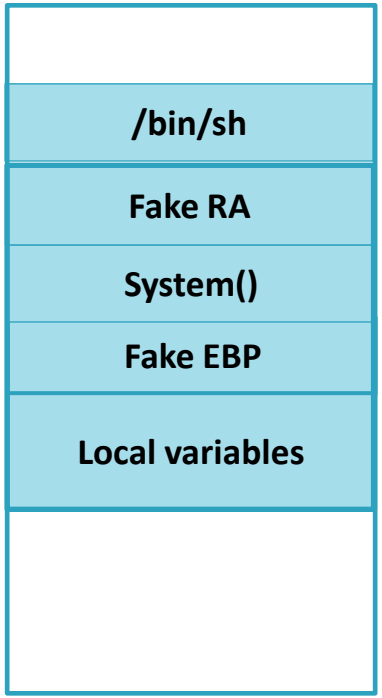
Return to libc in action…

- The executions of assembly instructions at the end of the vulnerable function will activate function *system()* with the *needed* argument!

```
mov %ebp,%esp
pop %ebp
ret
```

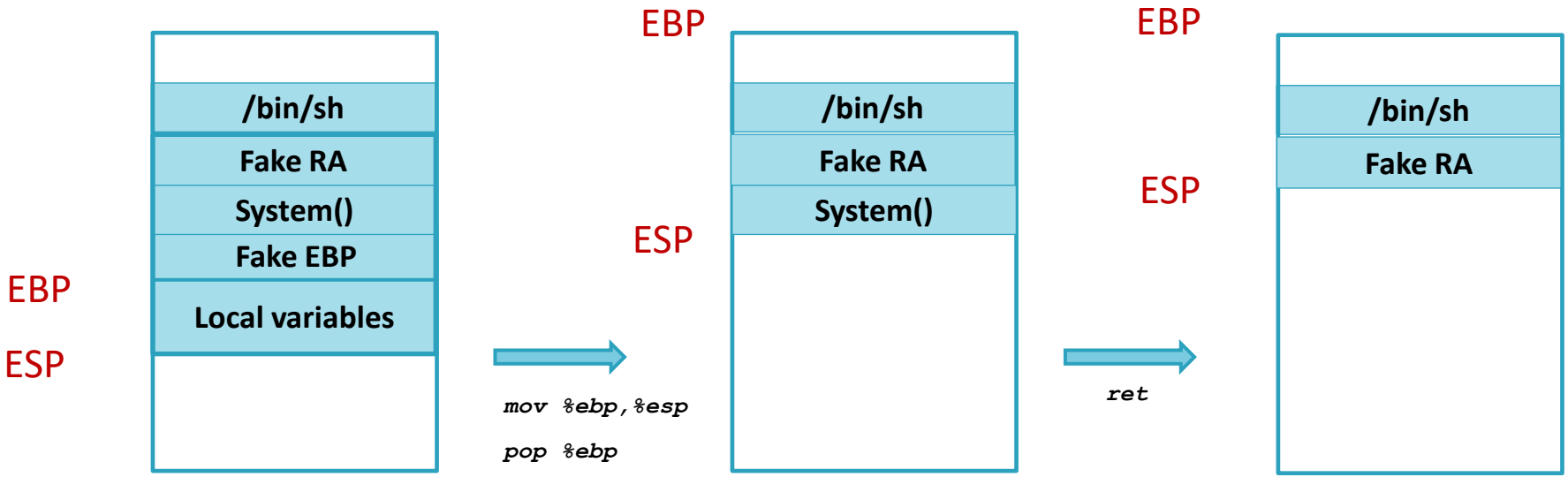
EBP

ESP



© CINI – 2021

Return to libc in action…



System(/bin/sh) is executed!

© CINI – 2021

Return to libc in action…

- To perform an attack we have to pass to a *vulnerable* program the following inputs:
  - A sequence of bytes long enough to trigger the buffer overflow (in our case 16)
  - The address of function *system()*
  - A sequence of 4 bytes for a return address
    - The address of function *exit() can be used.*
  - *The address of /bin/sh*

© CINI – 2021

Return to *libc*

Michele LORETI
Università di Camerino





https://cybersecnatlab.it